# Dimostrazioni degli alberi rosso-nero, rod cutting

| | |
|---|---|
| Materia | Algoritmi |
| Anno | Secondo Anno |
| Data | @November 21, 2024 |
| Studiato | Done |

Abbiamo già visto le proprietà degli alberi rosso-nero. Ricordiamo l'altezza nera $bh(x)$ come il numero di nodi neri che si incontrano in un cammino dal nodo $x$ ad una delle sue foglie. Essa è ben definita dalla proprietà $5$. L'altezza $h(x)$ di un nodo $x$ è il numero di nodi in un cammino da $x$ alle foglie. L'altezza nera di un albero è l'altezza nera dalla radice.

## Lemma

Vale il seguente lemma:

> *Un albero rosso-nero con $n$ nodi interni ha altezza al più $2\log(n+1)$, cioè $O(\log n)$.*

Dimostriamo questo lemma a partire dalla seguente osservazione:

> *Il sottoalbero radicato al nodo $x$ contiene almeno $2^{bh(x)}-1$ nodi interni.*

Dimostriamo l'osservazione per induzione sull'altezza $h(x)$ di $x$. Consideriamo la base dell'induzione con $h(x)=0$. Se $h(x)=0 \implies x$ è una foglia, quindi il sottoalbero radicato a $x$ ha $0$ sottoalberi. Se l'altezza è $0$, allora anche l'altezza nera è pari a $0$. Infatti:

$$h(x)=0 \implies bh(x)=0 \implies 0 \geq 2^{bh(x)}-1=2^0-1=1-1=0$$

Dimostriamo il passo induttivo. Assumiamo che $h(x)>0$ e che l'osservazione sia vera per i nodi di altezza inferiore ad $h(x)$.
Poiché

$h(x) > 0 \implies x$ ha due figli (**uno di essi può anche essere nullo**). Consideriamo un figlio di $x$:

- se il figlio è nero, $bh(x) = bh(figlio) + 1$;
- se il figlio è rosso, $bh(x) = bh(figlio)$.

Definitivamente possiamo concludere che $bh(figlio) \geq bh(x) - 1$.
Possiamo dire che
$h(figlio) < h(x) \implies$ per il nodo figlio vale l'ipotesi induttiva quindi il sottoalbero radicato al nodo figlio di $x$ contiene un numero di nodi maggiore o uguale a $2^{bh(figlio)} - 1$.
Ora consideriamo il numero di nodi interni al sottoalbero radicato a $x$. Tale numero è pari a $1+$ il numero di nodi del sottoalbero radicato al figlio di $x$ moltiplicato per 2. Chiamiamo $N$ il numero di nodi del sottoalbero radicato al figlio di $x$. Allora:

$$1 + N \cdot 2 \geq 1 + (2^{bh(figlio)} - 1) \cdot 2 \geq 1 + 2(2^{bh(x)-1} - 1) = 1 + 2^{bh(x)} - 2 = 2^{bh(x)} - 1$$

Quindi l'osservazione è dimostrata. Utilizzeremo l'osservazione per dimostrare il lemma.

Indichiamo con $n$ il numero di nodi interni dell'albero. L'altezza $h^*$ dell'albero è uguale all'altezza della radice. Sappiamo che $n \geq 2^{bh(root)} - 1$. L'altezza nera della radice è $bh(root) \geq \frac{h^*}{2}$: questo perché il numero di nodi neri è almeno pari a metà dei nodi di un qualunque cammino per la proprietà $4$. Quindi si ha che:

$$n \geq 2^{bh(root)} - 1 \geq 2^{\frac{h^*}{2}} - 1 \implies 2^{\frac{h^*}{2}} \leq n + 1 \implies \frac{h^*}{2} \leq \log(n+1) \implies$$
$$\implies h^* \leq 2\log(n+1)$$

# Problemi di ottimizzazione

Un problema di ottimizzazione differisce da un problema decisionale nel fatto che nel problema decisionale dobbiamo solo trovare una soluzione per un problema in base a dei requisiti stretti. Nel problema di ottimizzazione abbiamo anche delle preferenze modellate da una funzione di costo. Quindi è necessario risolvere il problema in base ai requisiti e ottimizzare la funzione di costo.

> *Un problema di ottimizzazione è un problema computazionale sulle cui variabili sono definite una o più funzioni di costo. L'obiettivo è trovare una soluzione che:*
>
> - *soddisfi i requisiti del problema (**hard constraints**);*

> - *ottimizzi o minimizzi una **funzione obiettivo** condizionata dalla **funzione di costo** (**soft constraints**).*
>
> *Una soluzione che soddisfa i requisiti del problema e minimizza o massimizza la funzione obiettivo è una **soluzione ottima**.*

## Programmazione dinamica

Per applicare la programmazione dinamica ad un problema di ottimizzazione, è necessario che tale problema:

- abbia una **struttura ricorsiva**;
- goda della **proprietà di sottostruttura ottima**;
- abbia un **numero polinomiale di sottoproblemi che si ripetono**, così che memorizziamo i risultati e miglioriamo la complessità.

Per quest'ultima proprietà, è necessaria una **dimostrazione formale**.

### Sottostruttura ottima

> *Un problema di ottimizzazione gode della proprietà di **sottostruttura ottima** se le restrizioni di una soluzione ottima ai sottoproblemi sono ottime per i sottoproblemi.*

### Esempio: Rod-Cutting

Per la richiesta del problema, vedi lezione faro.

La risoluzione del problema ricorsivo è:

$$r_n = \max\{ p_n, r_1 + r_{n-1}, ..., r_{\lceil \frac{n}{2} \rceil}, r_{\lfloor \frac{n}{2} \rfloor} \}$$

o meglio:

$$r_n = \max\{ p_i + r_{n-1} \mid 1 \leq i \leq n \} \;\; r_0 = 0$$

Ogni iterazione di una istanza genera un solo sottoproblema.
Un algoritmo ricorsivo impiega un tempo
$O(2^n)$. Una soluzione basata su programmazione dinamica, invece, impiega $O(n^2)$.

Proviamo che vale la proprietà di sottostruttura ottima per questo problema usando una **dimostrazione per assurdo**.

Sia
$r_n^*$ una soluzione ottima al problema, ossia che $r_n^* \geq r_n \ \ \forall n$. Supponiamo che $r_n^* = p_i + r_{n-i}^*$. Supponiamo per assurdo che $r_{n-1}^*$ **non sia ottimo**. Allora significa che:

$$\exists r'_{n-i} > r^*_{n-i} \implies r'_n = p_i + r'_{n-i}$$

Allora ciò significa che $r'_n = p_i + r'_{n+i} > p_i + r^*_{n-i} = r^*_n$, ma ciò è un assurdo, perché $r_n^*$ per ipotesi è una soluzione ottima. Quindi dobbiamo supporre che anche $r^*_{n-i}$ sia ottimo e quindi significa che il problema gode della proprietà di sottostruttura ottima.